CREDENCE RESOURCE MANAGEMENT (CRM)

iResolve Platform

SAST / DAST Security Testing Module

Technical Specification — v1.0

Multi-Engine Static & Dynamic Application Security Testing

Prepared for: iResolve Platform Team
Author: Karthik — Data & AI Engineer, Credence Resource Management
Date: July 2026

Status: Draft for Review

Table of Contents

1. Overview & Objective
2. Scope
3. Architecture Overview
4. SAST Engine Stack
5. DAST Engine Stack
6. Correlation & Confidence Layer (custom-built)
7. Data Model (PostgreSQL)
8. API Endpoints (FastAPI)
9. Frontend (React)
10. Reporting & Export
11. Compliance Mapping Reference (sample)
12. Phased Implementation Roadmap
13. Open Questions

1. Overview & Objective

This module adds enterprise-grade Static and Dynamic Application Security Testing (SAST/DAST) capability directly into iResolve. It is designed to detect the same technical vulnerability classes covered by commercial platforms (Checkmarx, Veracode, Snyk Code) using a layered stack of open-source engines, correlated and reported through a custom analysis layer built in-house.

- **Important compliance note:** this module produces findings mapped to OWASP Top 10, CWE, and PCI-DSS/HIPAA technical safeguard categories. It is compliance-aligned, not compliance-certified — a licensed commercial scanner is still required if a formal PCI-DSS/HIPAA attestation report needs to be signed off by an external auditor.

1.1 Goals

- Detect security vulnerabilities in iResolve's own codebase (FastAPI backend + React frontend) via SAST.
- Allow on-demand DAST scans against ad-hoc external target URLs submitted by users.
- Correlate findings across multiple scanning engines to reduce false positives and raise confidence.
- Map every finding to CWE, OWASP Top 10, and PCI-DSS/HIPAA reference tables.
- Provide a dashboard with severity heatmaps, trend tracking, and exportable DOCX/PDF reports.
- Reuse iResolve's existing architecture patterns: FastAPI, Celery/Redis async jobs, PostgreSQL, module-based permissions.

1.2 Non-Goals

- This module does not replace a licensed compliance-certified scanner for formal audit sign-off.
- This module does not perform penetration testing beyond automated DAST crawling/active scanning (no manual exploitation).
- This module does not scan third-party/client production systems without explicit authorization workflows (see Section 8).

2. Scope

Target Type	Method	Trigger
iResolve internal repository (FastAPI + React)	SAST — source code analysis	On-demand + scheduled (nightly)
Ad-hoc external URLs (client-facing apps, staging envs)	DAST — runtime black-box scan	On-demand only
Dependencies (npm / pip packages)	SCA — software composition analysis	On-demand + scheduled
Committed secrets / credentials	Secret scanning	On every scan run (bundled with SAST)

3. Architecture Overview

The module follows the same async job pattern already used by the RCA/CAPA and QA modules: FastAPI receives the scan request, validates and queues it, Celery workers execute the actual scanner processes, and results are normalized into PostgreSQL for the dashboard to query.

3.1 High-Level Flow

- 1. User submits a scan request via React UI — either a Git repo/path (SAST) or a target URL (DAST).
- 2. FastAPI validates input, checks module permission (require_module), creates a scan_run record (status = queued).
- 3. Celery task dispatched to the appropriate worker queue (sast_queue or dast_queue).
- 4. Worker orchestrates the relevant engines sequentially or in parallel, captures raw output (JSON/SARIF).
- 5. Normalization layer parses each engine's output into a common finding schema.
- 6. Correlation engine deduplicates and cross-references findings, assigns confidence score.
- 7. Findings persisted to PostgreSQL; scan_run status updated to complete; frontend notified via polling or WebSocket.
- 8. Dashboard renders results; user can export a report.

3.2 Component Diagram (textual)

```
React Frontend -> FastAPI (/scans) -> Redis (queue) -> Celery Workers
                                                       |-- SAST Orchestrator
                                                       |     |-- Semgrep
                                                       |     |-- CodeQL
                                                       |     |-- Bandit
                                                       |     |-- ESLint-security
                                                       |     |-- Gitleaks
                                                       |     `-- Trivy (SCA)
                                                       `-- DAST Orchestrator
                                                             `-- OWASP ZAP (daemon)
                                    |
                              Normalization + Correlation Layer
                                    |
                               PostgreSQL (findings store)
                                    |
                              React Dashboard <- FastAPI (/findings)
```

4. SAST Engine Stack

Engine	Role	Coverage
Semgrep	Pattern-based static analysis	OWASP Top 10, CWE rulesets, custom Artiva/FastAPI rules
CodeQL	Semantic / dataflow (taint) analysis	Cross-function source-to-sink tracing — the depth layer Checkmarx-class tools rely on
Bandit	Python AST-level analysis	FastAPI backend-specific issues (SQL injection, insecure deserialization, hardcoded secrets)
ESLint + eslint-plugin-security	JS/JSX static analysis	React frontend — XSS sinks, unsafe eval, insecure regex

Engine	Role	Coverage
Gitleaks	Secret scanning	API keys, DB credentials, tokens committed to history
Trivy	Software composition analysis (SCA)	Vulnerable npm/pip dependencies, optional container image scanning

4.1 Orchestration Notes

- Each engine runs as an isolated subprocess inside the Celery worker container, writing results to a temp directory scoped to the scan_run ID.
- Engines that support SARIF output (Semgrep, CodeQL) should use it as the common intermediate format where possible.
- Run engines in parallel where resource limits allow (Celery worker concurrency), rather than sequentially, to reduce total scan time.
- CodeQL requires a pre-built database per language (codeql database create) — cache this per repo/commit to avoid rebuilding on every scan.

5. DAST Engine Stack

Component	Role
OWASP ZAP (daemon mode)	Core DAST engine — spider + active scan against target URL
ZAP REST API	FastAPI backend controls scan lifecycle: start, poll status, pull alerts
Authentication handler	Optional login sequence recorder for scanning authenticated app areas

5.1 DAST Flow Detail

- User submits target URL + optional auth context (login URL, credentials, session token) via a securely stored scan profile.
- FastAPI calls ZAP's /JSON/spider/action/scan/ to crawl the target, then /JSON/ascan/action/scan/ for the active scan.
- Worker polls /JSON/ascan/view/status/ until 100% complete (this can take minutes — always async, never a blocking request).
- Results pulled via /JSON/core/view/alerts/ and normalized into the common finding schema.
- **Authorization guardrail:** before scanning any external target, the UI must require the user to confirm they have authorization to test that target. Log the confirmation with the scan_run record — unauthorized active scanning against third-party systems carries real legal risk (analogous to unauthorized penetration testing).

6. Correlation & Confidence Layer (custom-built)

This is the differentiator that makes the module feel enterprise-grade rather than "just running Semgrep." It is built in-house, not wrapped from an open-source tool.

6.1 Deduplication

- Findings are grouped by CWE ID + file path + line number (SAST) or CWE ID + endpoint + parameter (DAST).
- If 2+ engines report the same group, they are merged into a single finding with a combined evidence list.

6.2 Confidence Scoring

Condition	Confidence Level
Flagged by 1 engine only, pattern-based	Low
Flagged by 1 engine with dataflow evidence (CodeQL)	Medium
Flagged by 2+ independent engines	High
Flagged by 2+ engines AND touches a PHI-tagged module	Critical — auto-escalated

6.3 Business-Context Tagging

- Findings inside modules known to touch PHI (per the existing HIPAA PHI scrubber work) are automatically tagged phi_context = true and their severity is escalated one level.
- A static module-to-sensitivity mapping table (config-driven) lets you mark which iResolve modules are PHI/PCI-relevant, so new modules just need a config entry, not code changes.

6.4 False-Positive Suppression

- Suppression rules stored per rule-ID + file-path pattern, reviewable/editable by an admin — same pattern as the QA module's manually-reviewed approach given Artiva/COS context gaps.
- Suppressed findings remain in the DB (soft-suppressed) with an audit trail of who suppressed them and why, not hard-deleted.

7. Data Model (PostgreSQL)

7.1 Core Tables

Table	Key Columns
scan_runs	id, scan_type (sast/dast), target (repo path or URL), status, requested_by, started_at, completed_at
scan_engine_results	id, scan_run_id (FK), engine_name, raw_output (JSONB), exit_code
findings	id, scan_run_id (FK), cwe_id, owasp_category, severity, confidence, file_path, line_number, endpoint, description, remediation, phi_context (bool)
finding_sources	id, finding_id (FK), engine_name, raw_evidence (JSONB) — supports the merged evidence list
suppressions	id, finding_rule_id, path_pattern, suppressed_by, reason, created_at
compliance_mappings	cwe_id, owasp_category, pci_dss_requirement, hipaa_safeguard — static reference table

7.2 Notes

- findings.severity follows CVSS-derived buckets (Critical/High/Medium/Low/Info); store the numeric CVSS score alongside the bucket for sorting.
- Use the same atomic-swap staging pattern already used for Artiva pulls if scan_engine_results grows large enough to need partitioned refresh — unlikely at initial scale but worth keeping consistent.

8. API Endpoints (FastAPI)

Method & Path	Purpose
POST /scans/sast	Queue a new SAST scan for a repo path/branch
POST /scans/dast	Queue a new DAST scan for a target URL (requires authorization confirmation flag)
GET /scans/{scan_run_id}	Poll scan status
GET /findings? scan_run_id=&severity=&phi_context=	List/filter findings
PATCH /findings/{id}/suppress	Suppress a finding with reason (admin/module-permission gated)
GET /findings/{id}	Full detail incl. merged evidence from all engines
GET /reports/{scan_run_id}/export	Generate DOCX/PDF report for a scan run
GET /compliance-mappings	Reference table for CWE -> OWASP -> PCI/HIPAA

All endpoints sit behind the existing require_module() dependency, gated on a new "security_scanner" module permission, consistent with the per-user module permission system already in place.

9. Frontend (React)

9.1 Screens

- Scan Launcher — tabbed form: "Scan Repository" (SAST) vs "Scan URL" (DAST), with the DAST tab requiring an authorization checkbox before submission.
- Live Scan Status — progress indicator per engine (queued/running/complete), using the site-wide WorkingIndicator component for consistency with the rest of iResolve.
- Findings Dashboard — severity heatmap, filterable table (severity, CWE, OWASP category, PHI context, confidence), trend chart (findings over time / mean-time-to-remediate).
- Finding Detail View — merged evidence from each engine, CWE/OWASP/PCI/HIPAA mapping, remediation guidance, suppress action.
- Suppression Admin — review/approve suppressions, audit log.

9.2 Design Language

Follows the same sci-fi/glassmorphism direction as the "Meet the Team" page — severity badges as glowing chips (Critical = red glow, High = amber, Medium = yellow, Low = green), dark-mode-first dashboard suited to a security tool.

10. Reporting & Export

- Reuses the existing docx (npm) + LibreOffice-headless pipeline established for resumes and spec docs.
- Report includes: executive summary, severity breakdown chart, full findings table with CWE/OWASP/PCI-DSS/HIPAA mapping, remediation appendix.
- PDF export via the same DOCX-to-PDF LibreOffice headless conversion already standardized across iResolve.

11. Compliance Mapping Reference (sample)

CWE	OWASP Category	PCI-DSS Ref	HIPAA Safeguard
CWE-89 (SQL Injection)	A03: Injection	Req 6.5.1	164.312(c) Integrity
CWE-79 (XSS)	A03: Injection	Req 6.5.7	164.312(e) Transmission Security
CWE-798 (Hardcoded Credentials)	A07: Auth Failures	Req 8.2.1	164.312(a) Access Control
CWE-327 (Broken Crypto)	A02: Crypto Failures	Req 4.1	164.312(a)(2)(iv) Encryption
CWE-611 (XXE)	A05: Security Misconfig	Req 6.5.1	164.312(c) Integrity

This table is seeded once and stored as compliance_mappings; extend it as new rule IDs are onboarded from each engine.

12. Phased Implementation Roadmap

Phase	Scope	Deliverable
Phase 1	SAST core: Semgrep + Bandit + ESLint-security + Gitleaks on iResolve's own repo	Working scan-on-demand, raw findings in DB, basic list UI
Phase 2	Add CodeQL + Trivy (SCA); build correlation/dedup + confidence scoring layer	Merged, deduplicated findings with confidence levels
Phase 3	DAST via OWASP ZAP for ad-hoc external URLs, authorization guardrail workflow	On-demand URL scanning live
Phase 4	Compliance mapping table, PHI-context tagging, severity dashboard, trend charts	Full dashboard parity with enterprise tool UX
Phase 5	DOCX/PDF report export, suppression admin workflow, scheduled nightly SAST/SCA scans	Production-ready module

13. Open Questions

- Should DAST scans against client/third-party Artiva-connected systems require a separate sign-off workflow beyond the checkbox (e.g., manager approval)?

- Where should the CodeQL database cache live given no-Git-on-HIPAA-server constraint — does this module warrant an exception, or should CodeQL run in a separate non-PHI build environment?
- Retention policy for raw_output JSONB in scan_engine_results — how long to keep full engine output vs. just normalized findings?
- Should suppressed findings still count toward the trend/dashboard metrics, or be excluded entirely?